



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Introduction to LLMs

ONE LOVE. ONE FUTURE.

1. Introduction to the course
2. NLP and LLMs
3. Language Models
4. Word tokenization

A graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern of red dots. The dots are arranged in concentric, slightly irregular rings, creating a sense of depth and movement. In the center of this pattern, the word "HUST" is written in a bold, white, sans-serif font.

HUST

Intro the the course

Điểm thành phần Module	Phương pháp đánh giá cụ thể Evaluation method	Mô tả Detail	CĐR được đánh giá Output	Tỷ trọng Percent
[1]	[2]	[3]	[4]	[5]
A1. Điểm quá trình Mid-term (*)	Đánh giá quá trình Progress evaluation			40%
	A1.1. Bài tập về nhà Homework	Tự luận Written	M1, M2;	10%
	A1.2. Bài tập lớn Capstone project	Báo cáo Presentation	M1, M3;	30%
A2. Điểm cuối kỳ Final term	A2.1. Thi cuối kỳ Final exam	Thi viết Written exam	M1÷M2	60%

** Điểm quá trình sẽ được điều chỉnh bằng cách cộng thêm điểm chuyên cần. Điểm chuyên cần có giá trị từ -2 đến +1, theo Quy chế Đào tạo đại học hệ chính quy của Trường ĐH Bách khoa Hà Nội.*

The evaluation about the progress can be adjusted with some bonus. The bonus should belong to [-2, +1], according to the policy of Hanoi University of Science and Technology.

Contents

- Introduction
- Different types of large language models
- Parameter-Efficient Fine-Tuning
- Continued pretraining of large language models
- Supervised fine-tuning
- LLM alignment
- Prompt Engineering with LLMs
- Long-context LLMs
- LLM evaluation
- Mitigation of Bias and Toxicity
- Recent advances in large language

Reference courses

1. [CS324: Large Language Models, Stanford University](#)
2. [COS 597G: Understanding Large Language Models, Princeton University](#)
3. [263-5354-00L: Large Language Models, ETH Zürich](#)
4. [CSC 6201/CIE 6021: Large Language Models, School of Data Science, The Chinese University of Hong Kong](#)
5. [CS 194/294-267: Understanding Large Language Models: Foundations and Safety, Berkley University](#)
6. [CS 324: Advances in Foundation Models, Stanford University](#)
7. [CS224N: Natural Language Processing with Deep Learning, Stanford University](#)
8. [6.5940: TinyML and Efficient Deep Learning Computing, MIT](#)
9. [CS25: Transformers United V4, Stanford University](#)
10. [CS 601.471/671 NLP: Self-supervised Models, John Hopkins University](#)



HUST

NLP and LLMs

What is a NLP?

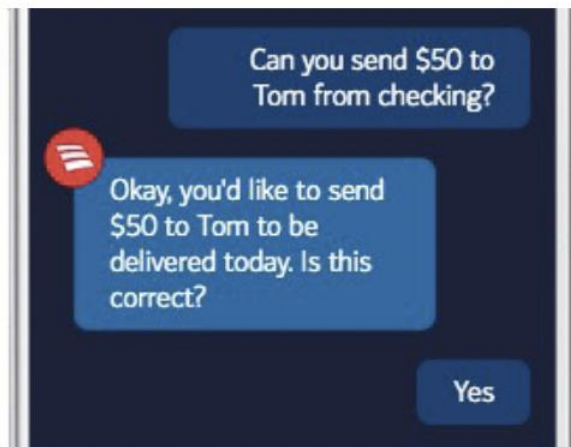
- NLP = building **computer programs** to analyze, understand and generate **human language - either spoken or written** (informal)
- NLP is an interdisciplinary field



What is a NLP?

- NLP = building **computer programs** to analyze, understand and generate **human language - either spoken or written** (informal)

Communication with humans (ex.
personal assistants, customer service)



Banking assistant

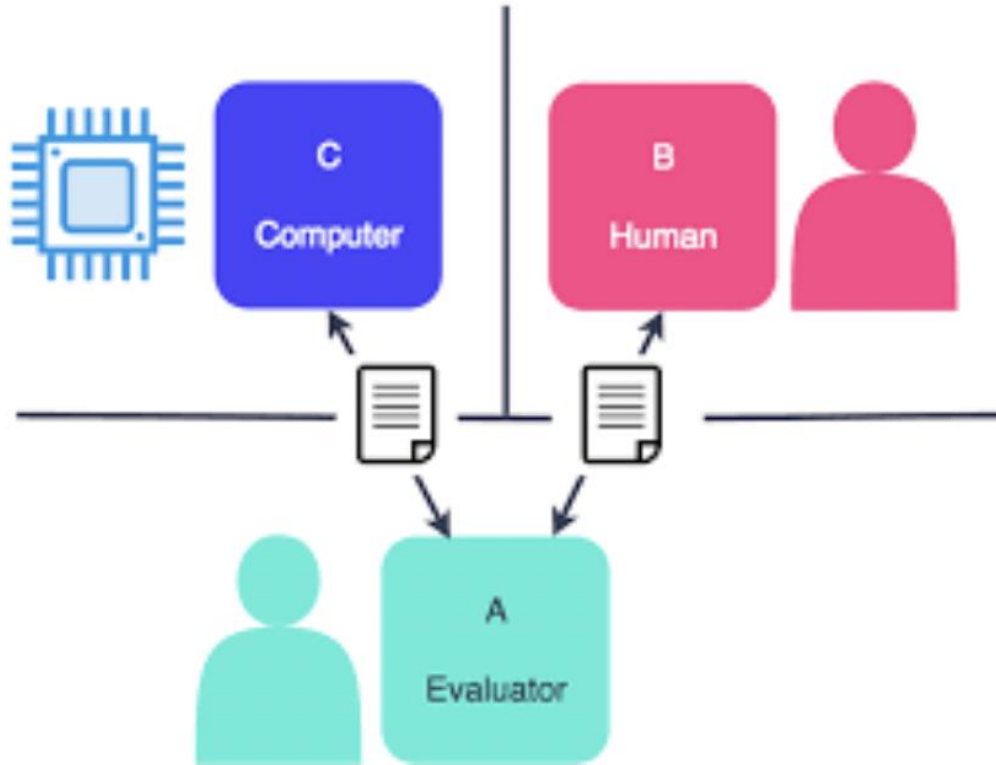
ONLINE

Access the wealth of information about
the world — crucial for AI systems



OFFLINE

Turing test



A. M. Turing (1950) *Computing Machinery and Intelligence*. *Mind* 49: 433-460.

COMPUTING MACHINERY AND INTELLIGENCE

By A. M. Turing

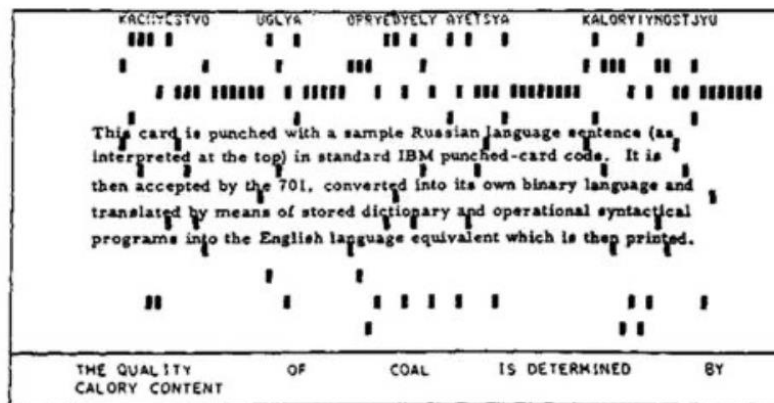
1. The Imitation Game



Ability to understand and generate language ~ intelligence

Machine translation

How it started

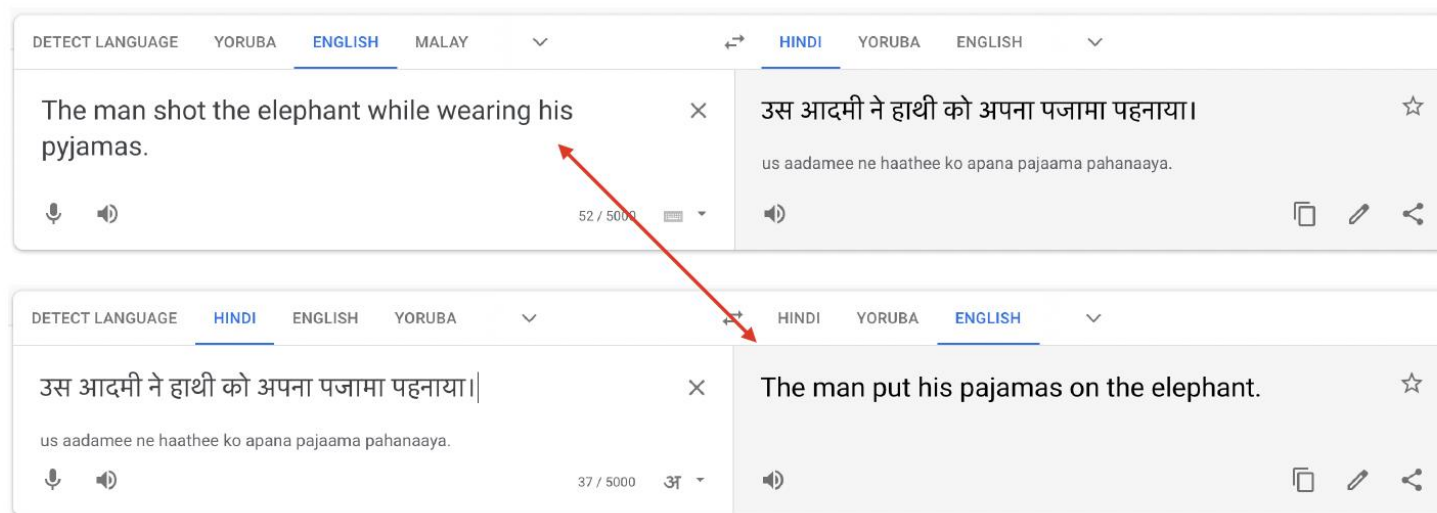


Specimen punched card and below a strip with translation, printed within a few seconds

Georgetown experiment
1954

“Within three or five
years, machine
translation will be a
solved problem”

How it's going



Machine translation

The image displays two screenshots of the Google Translate web interface, illustrating the process of machine translation between English and Hindi.

Top Screenshot: The interface shows the 'Text' tab selected. The source language is set to 'ENGLISH' and the target language is 'HINDI'. The input text is 'The man shot the elephant while wearing his pajamas'. The translated Hindi text is 'आदमी ने अपना पजामा पहने हुए हाथी को गोली मार दी'. Below the Hindi text, the phonetic transcription 'aadamee ne apana pajaama pahane hue haathee ko golee maar dee' is provided. The character count '51 / 5,000' is visible.

Bottom Screenshot: The interface shows the 'Text' tab selected. The source language is set to 'HINDI' and the target language is 'ENGLISH'. The input text is 'आदमी ने अपना पजामा पहने हुए हाथी को गोली मार दी'. The translated English text is 'Man shoots elephant wearing his pajamas'. Below the English text, the phonetic transcription 'aadamee ne apana pajaama pahane hue haathee ko golee maar dee' is provided. The character count '47 / 5,000' is visible.

(Based on Google Translate result in 2023-01)

A General Framework for NLP Systems

- Create a function to map an **input** X into an **output** Y , where X and/or Y involve language.

Input X

Text

Text

Text

Text

Image

Output Y

Continuing Text

Text in Other Language

Label

Linguistic Structure

Text

Task

Language Modeling

Translation

Text Classification

Language Analysis

Image Captioning

Methods for Creating NLP Systems

- **Rules:** Manual creation of rules

```
def classify(x: str) -> str:
    sports_keywords = ["baseball", "soccer", "football", "tennis"]
    if any(keyword in x for keyword in sports_keywords):
        return "sports"
    else:
        return "other"
```

- **Prompting:** Prompting a language model w/o training

If the following sentences is about "sports"
reply "sports". Otherwise reply "other".

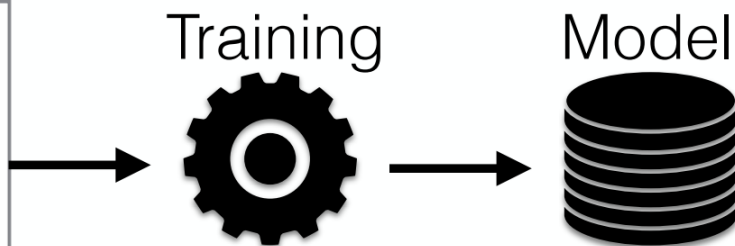
{X}

→ LM

- **Fine-tuning:** Machine learning from paired data $\langle X, Y \rangle$

I love to play baseball.
The stock price is going up.
He got a hat-trick yesterday.
He is wearing tennis shoes.

sports
other
sports
other

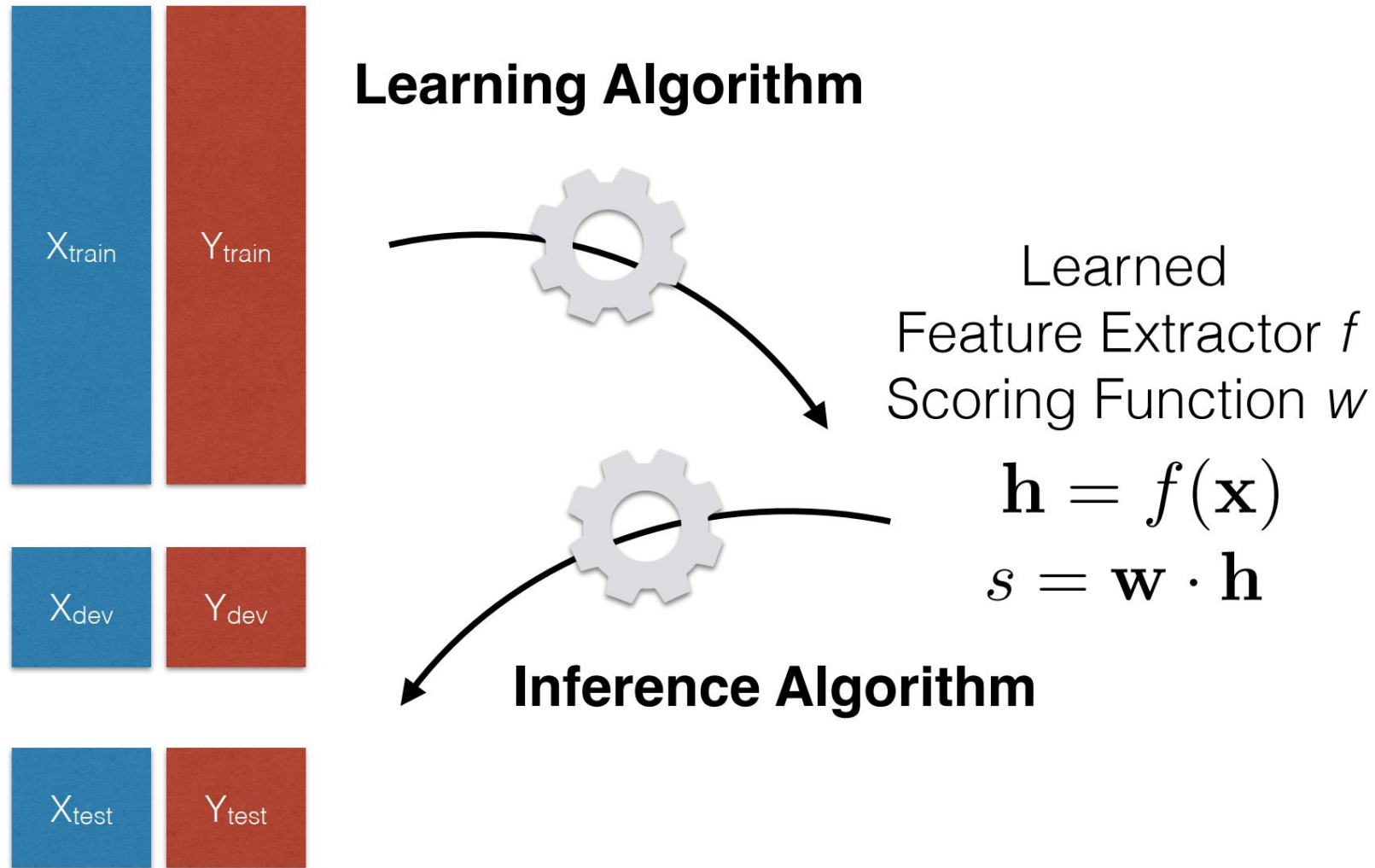


Data Requirements for System Building

- **Rules/prompting based on intuition:**
No data needed, but also no performance guarantees
- **Rules/prompting based on spot-checks:**
A small amount of data with input X only
- **Rules/prompting with rigorous evaluation:**
Development set with input X and output Y (e.g. 200-2000 examples). Additional held-out test set also preferable.
- **Fine-tuning:**
Additional train set. More is often better — constant accuracy increase when data size doubles.



Machine learning



The era of deep learning

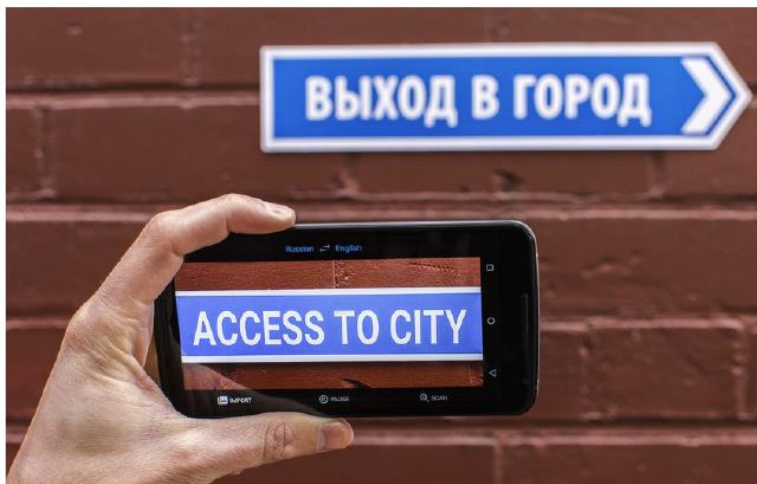
- Significant advances in core NLP technologies
- **Essential ingredient:** large-scale supervision, lots of compute
- Reduced manual effort - less/zero **feature engineering**



GPU



TPU



36M sentence pairs

Russian: Машинный перевод - это круто!

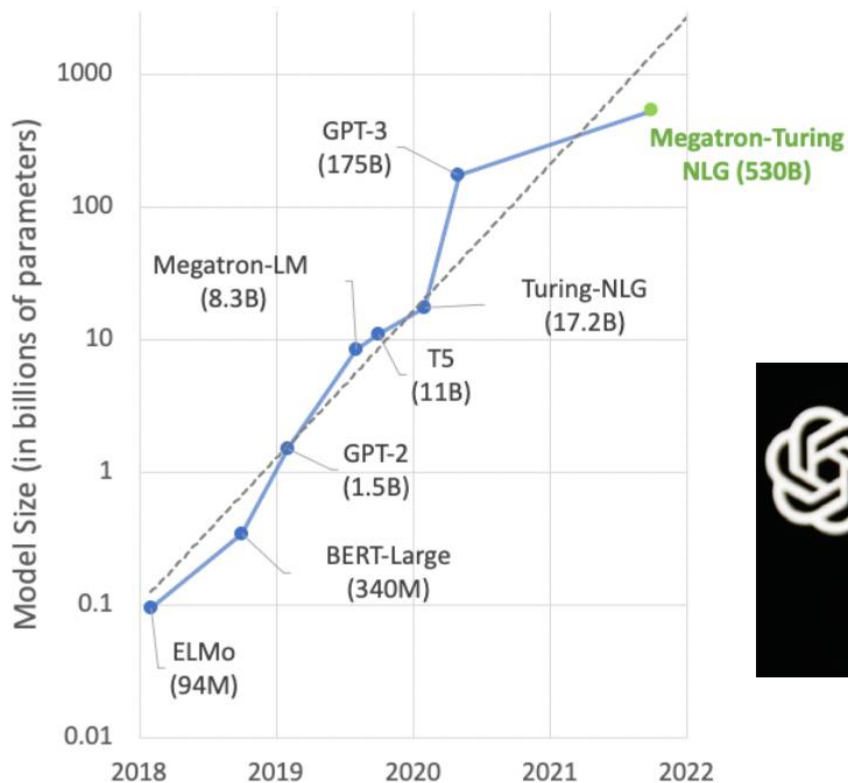


English: Machine translation is cool!

The era of pre-training and LLMs

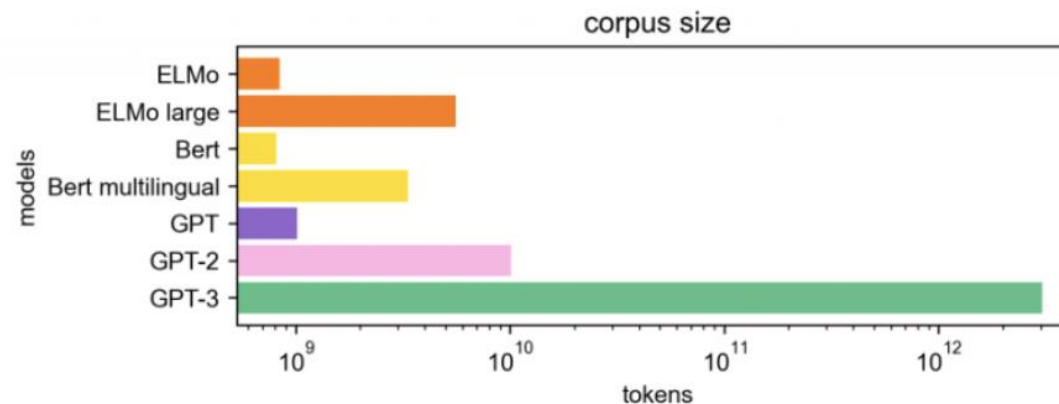
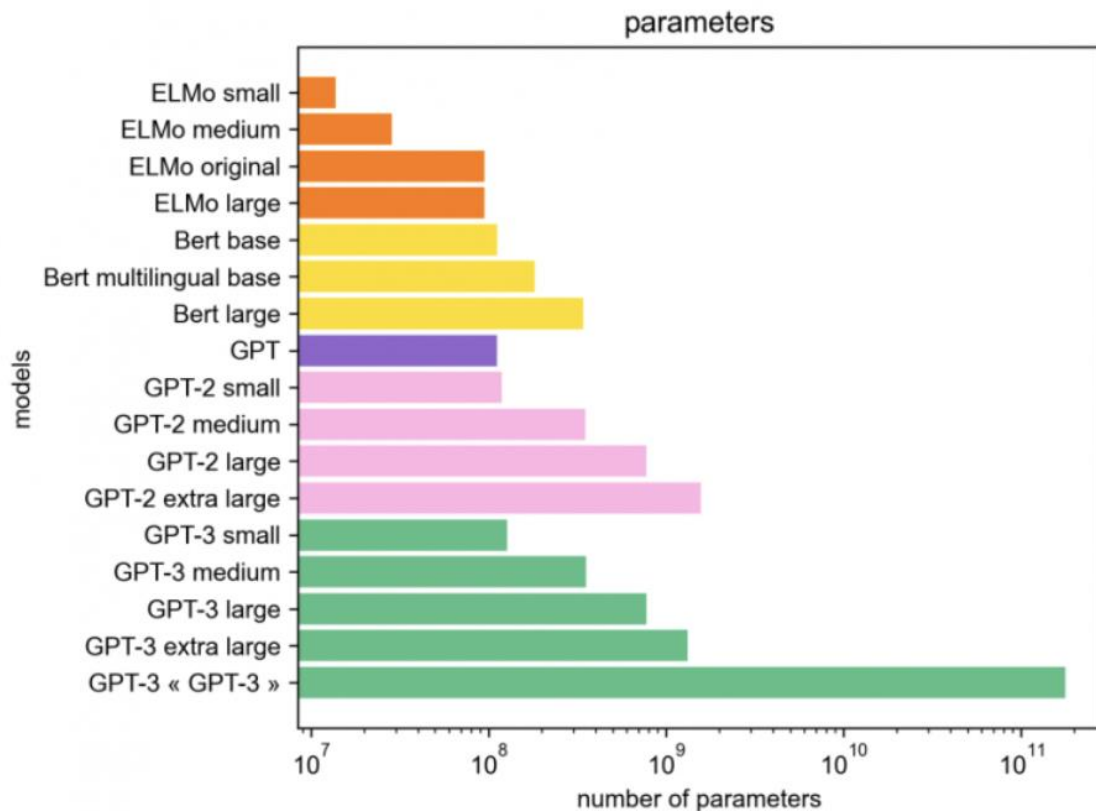


BERT, ELMo, ERNIE...



- Leverages a lot of unlabeled text
- Model size increased by $10^3 - 10^5$ x in parameters

How large are “large” LMs?



More recent models: PaLM (540B), OPT (175B), BLOOM (176B)...

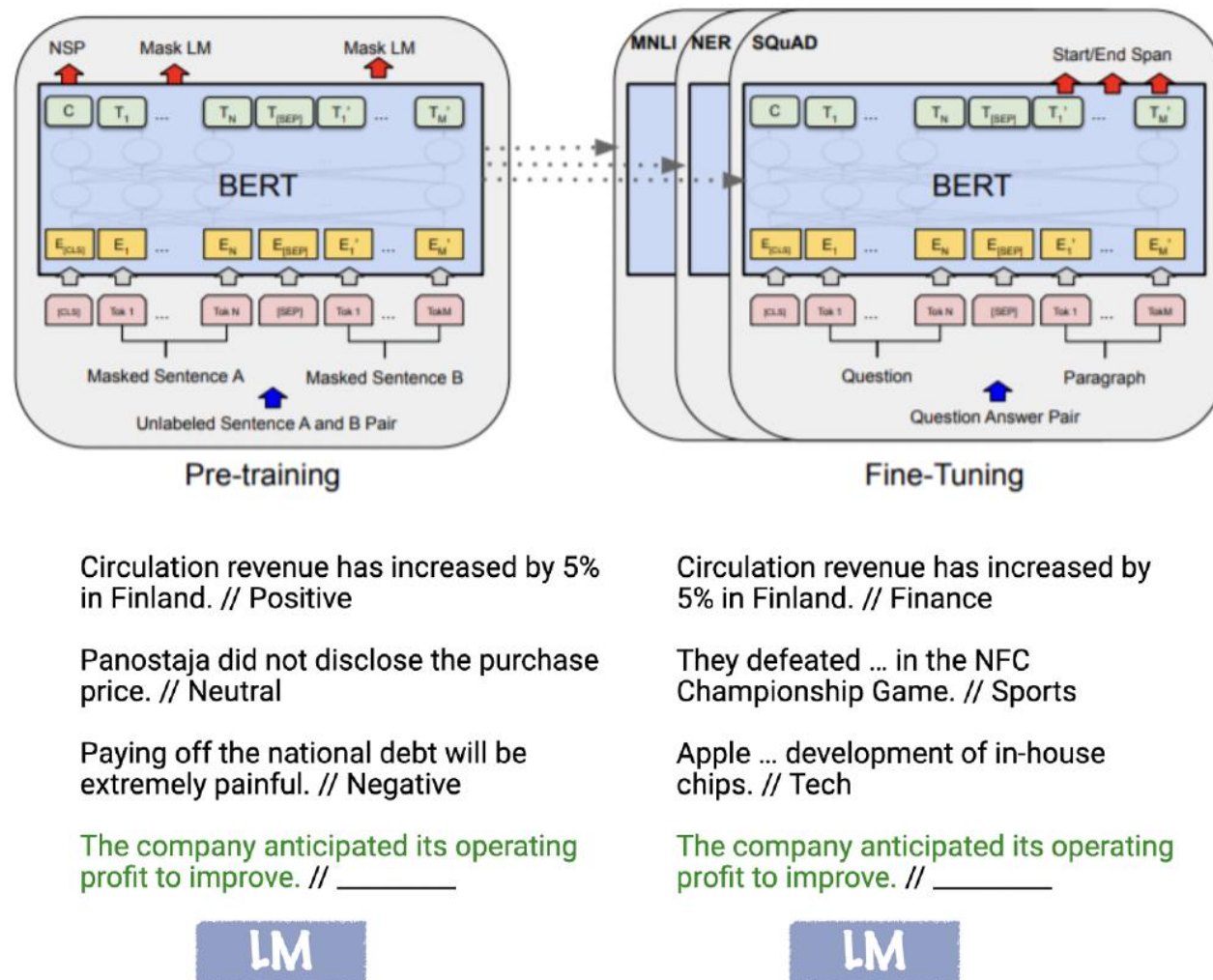
Image source: <https://hellofuture.orange.com/en/the-gpt-3-language-model-revolution-or-evolution/>

How large are “large” LMs?

- Today, we mostly talk about two camps of models:
 - Medium-sized models: BERT/RoBERTa models (100M or 300M), T5 models (220M, 770M, 3B)
 - “Very” large LMs: models of 100+ billion parameters
- Larger model sizes $\Rightarrow$ larger compute, more expensive during inference
- Different sizes of LMs have different ways to adapt and use them
 - Fine-tuning, zero-shot/few-shot prompting, in-context learning...
- Emergent properties arise from model scale
- Trade-off between model size and corpus size

Pre-training and adaptation

- **Pre-training:** trained on huge amounts of unlabeled text using “self-supervised” training objectives
- **Adaptation:** how to use a pre-trained model for your downstream task?
 - What types of NLP tasks (input and output formats)?
 - How many annotated examples do you have?



Why LLMs?

- The promise: one single model to solve many NLP tasks

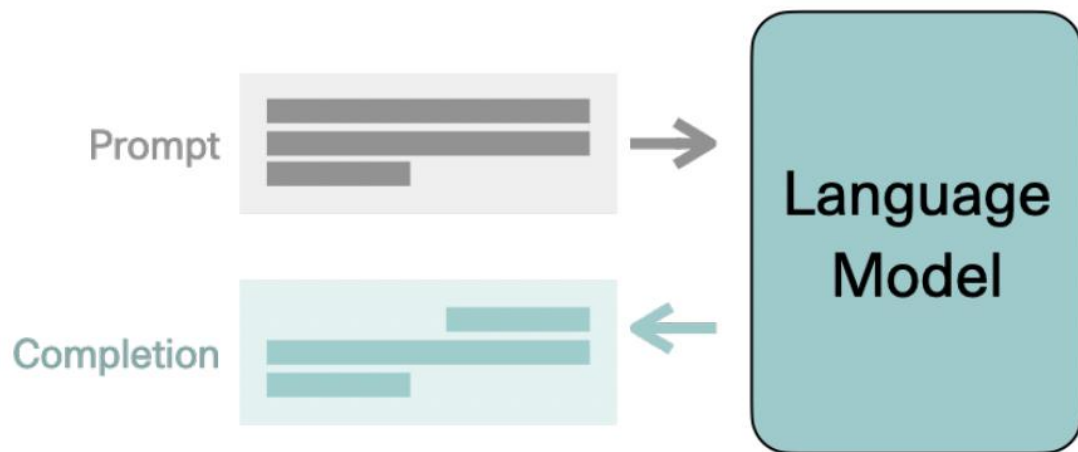
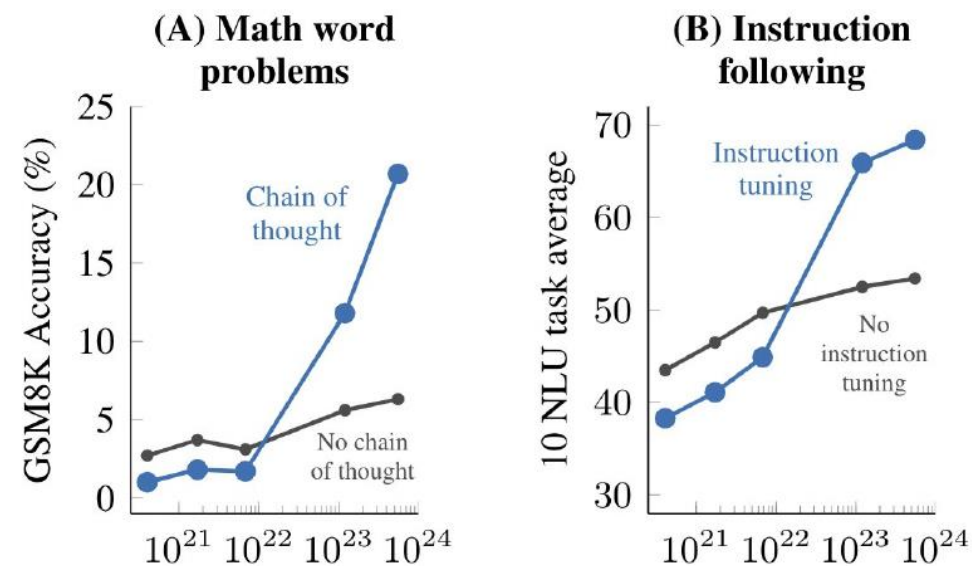


Image credit: Jay Alammar

- Emergent properties in LLMs



(Wei et al., 2022)



HUST

Language models

Generative vs. Discriminative Models

- **Discriminative model:** a model that calculates the probability of a latent trait given the data

mô hình phân biệt

$$P(Y | X)$$

conditional

- **Generative model:** a model that calculates the probability of the input data itself

mô hình tạo sinh

$$P(X)$$

stand-alone

$$P(X, Y)$$

joint

What is a LM?

$$P(X)$$



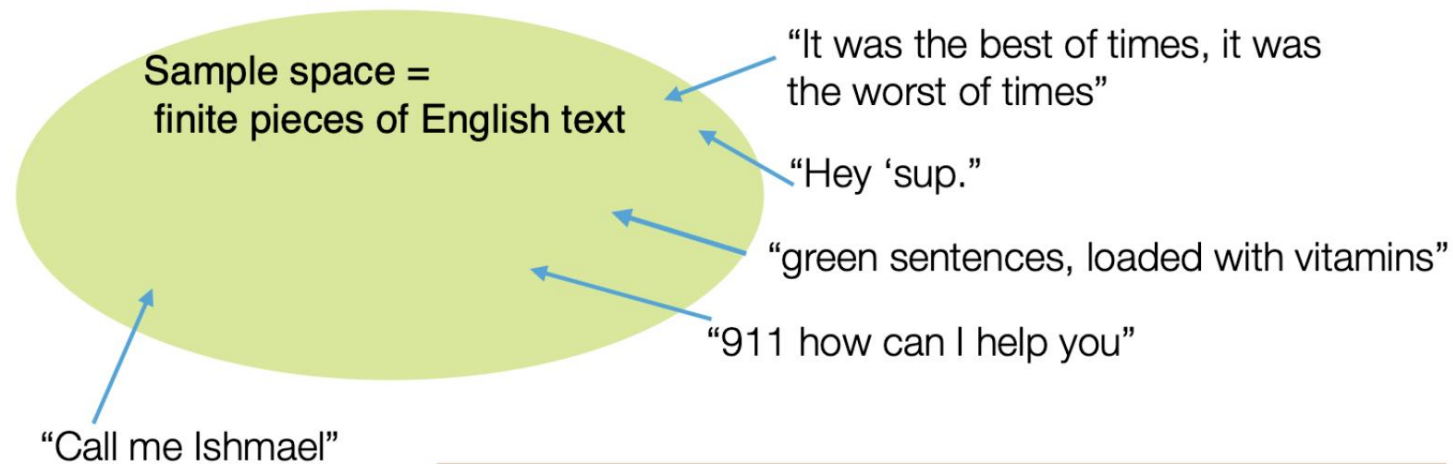
Sentence/Document

A generative model that calculates the probability of language

What is a LM?

- A probabilistic model of a sequence of words
- Joint probability distribution of words $w_1, w_2, \dots, w_n$:

$$P(w_1, w_2, w_3, \dots, w_n)$$



How likely is a given phrase, sentence, paragraph or even a document?

(i.e., $\Pr[w_1 w_2 w_3 \dots w_n]$ associated with every finite word sequence $w_1 w_2 w_3 \dots w_n$ (including nonsensical ones))

$$p(w_1, w_2, w_3, \dots, w_n) = p(w_1) p(w_2 | w_1) p(w_3 | w_1, w_2) \times \dots \times p(w_n | w_1, w_2, \dots, w_{n-1})$$

Conditional probability:
 $p(w | w_1, w_2), \forall w \in V$

Sentence: “the cat sat on the mat”

$$P(\text{the cat sat on the mat}) = P(\text{the}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{the cat}) \\ * P(\text{on}|\text{the cat sat}) * P(\text{the}|\text{the cat sat on}) \\ * P(\text{mat}|\text{the cat sat on the})$$

Implicit order

LMs are everywhere



how is the weather in new

- how is the weather in new **york**
- how is the weather in new **zealand**
- how is the weather in new **orleans**
- how is the weather in new **jersey**
- how is the weather in new **orleans in february**
- how is the weather in new **york in march**
- how is the weather in new **orleans in january**
- how is the weather in new **mexico**
- how is the weather in new **york in february**
- how is the weather in new **orleans in december**

Google Search I'm Feeling Lucky

Report inappropriate predictions

New Message

Cancel

To:



Language models are the



best

most

same

What Can we Do w/ LMs?

- **Score** sentences:

$P(\text{Jane went to the store .}) \rightarrow \text{high}$

$P(\text{store to Jane went the .}) \rightarrow \text{low}$

(same as calculating loss for training)

- **Generate** sentences:

$$\tilde{x} \sim P(X)$$

How Can we Apply These?

- **Answer questions**
 - *Score* possible multiple choice answers
 - *Generate* a continuation of a question prompt
- **Classify text**
 - *Score* the text conditioned on a label
 - *Generate* a label given a classification prompt
- **Correct grammar**
 - *Score* each word and replace low-scoring ones
 - *Generate* a paraphrase of the output

Estimating probability



$$P(\text{sat}|\text{the cat}) = \frac{\text{count}(\text{the cat sat})}{\text{count}(\text{the cat})}$$

trigram
bigram

$$P(\text{on}|\text{the cat sat}) = \frac{\text{count}(\text{the cat sat on})}{\text{count}(\text{the cat sat})}$$

⋮

Maximum
likelihood
estimate
(MLE)

Assume we have a vocabulary of size V ,
how many sequences of length n do we have?

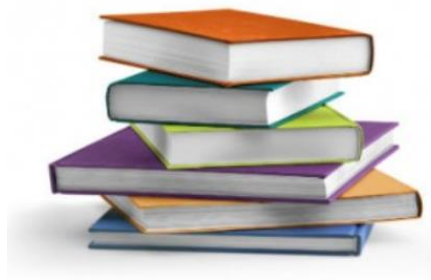
A) $n * V$

B) n^V

C) V^n

D) V/n

Estimating probability



$$P(\text{sat}|\text{the cat}) = \frac{\text{count}(\text{the cat sat})}{\text{count}(\text{the cat})}$$

$$P(\text{on}|\text{the cat sat}) = \frac{\text{count}(\text{the cat sat on})}{\text{count}(\text{the cat sat})}$$

⋮

Maximum
likelihood
estimate
(MLE)

- With a vocabulary of size V , # sequences of length $n = V^n$
- Typical English vocabulary ~ 40k words
- Even sentences of length ≤ 11 results in more than $4 * 10^{50}$ sequences.
Too many to count! (*# of atoms in the earth* ~ 10^{50})

Markov assumption

- Use only the recent past to predict the next word
- Reduces the number of estimated parameters in exchange for modeling capacity
- 1st order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat}|\text{the})$$

- 2nd order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat}|\text{on the})$$



Andrey Markov

kth order Markov assumption

Consider only the last k words (or less) for context

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

which implies the probability of a sequence is:

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

(assume $w_j = \phi \quad \forall j < 0$)

Need to estimate counts for up to $(k+1)$ grams

n-gram models

Unigram $P(w_1, w_2, \dots w_n) = \prod_{i=1}^n P(w_i)$ e.g. $P(\text{the}) P(\text{cat}) P(\text{sat})$

Bigram $P(w_1, w_2, \dots w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$ e.g. $P(\text{the}) P(\text{cat} | \text{the}) P(\text{sat} | \text{cat})$

and Trigram, 4-gram, and so on.

*Larger the n , more accurate and better the language model
(but also higher costs)*

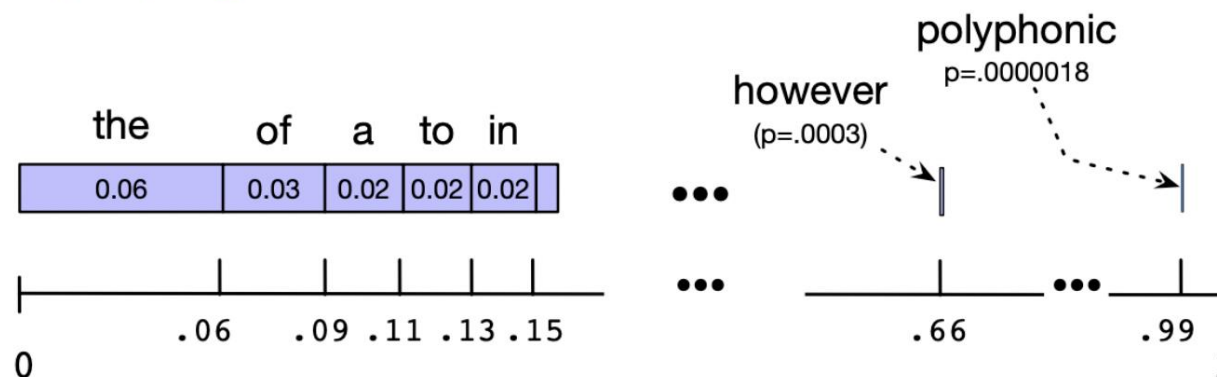
Caveat: Assuming infinite data!

Generating from a language model

- Given a language model, how to generate a sequence?

$$\text{Bigram } P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$$

- Generate the first word $w_1 \sim P(w)$
- Generate the second word $w_2 \sim P(w | w_1)$
- Generate the third word $w_3 \sim P(w | w_2)$
- ...



Generating from a language model

- Given a language model, how to generate a sequence?

Trigram
$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_{i-2}, w_{i-1})$$

- Generate the first word $w_1 \sim P(w)$
- Generate the second word $w_2 \sim P(w \mid w_1)$
- Generate the third word $w_3 \sim P(w \mid w_1, w_2)$
- Generate the fourth word $w_4 \sim P(w \mid w_2, w_3)$
- ...

Generations

Unigram

*release millions See ABC accurate President of Donald Will
cheat them a CNN megynkelly experience @ these word
out- the*

Bigram

*Thank you believe that @ ABC news, Mississippi tonight
and the false editorial I think the great people Bill Clinton
. "*

Trigram

*We are going to MAKE AMERICA GREAT AGAIN!
#MakeAmericaGreatAgain <https://t.co/DjkdAzT3WV>*

Typical LMs are not sufficient to handle long-range dependencies

Example from a GPT-2 output (2019): **prompt aka. conditional context**

With the start of the new academic year, Princeton has an opportunity to help provide a new generation of women with a diverse set of academic resources for higher education. We are offering the resources of the Princeton-McGill program specifically to women with undergraduate degrees who would like to enhance their academic experience. Princeton-McGill offers a comprehensive suite of services for women and their families including a variety of graduate programs, support programs, and the opportunity to serve as leaders in their communities with a wide variety of programs, activities and services. For the upcoming fall, Princeton-McGill will also offer its Women's Center , which is located in a renovated women's dorm. At Princeton, we are working with the Princeton-McGill community to develop a suite of programs that are designed to give new and returning students a strong foundation for a successful, rewarding graduate career. The Women's Center , the Princeton-McGill Women's Center provides a range of supports to address the specific needs of female doctoral degree graduates. Programs are tailored to meet the unique needs of women under the age of 28, women and families

<https://talktotransformer.com/>

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_{i-1024}, \dots, w_{i-2}, w_{i-1})$$

Modern LMs can handle much longer contexts!

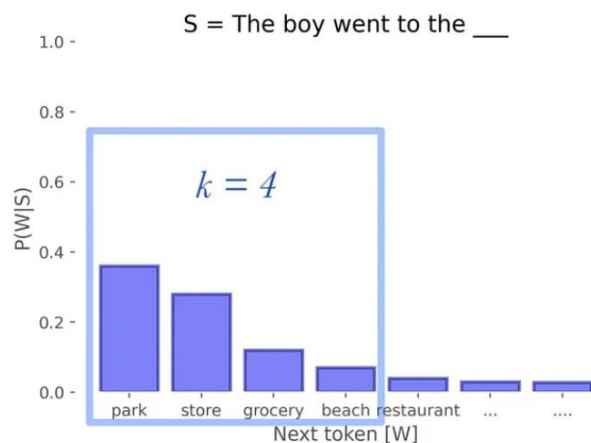
Generation methods (advanced)

- Greedy: choose the most likely word!

To predict the next word given a context of two words w_1, w_2 :

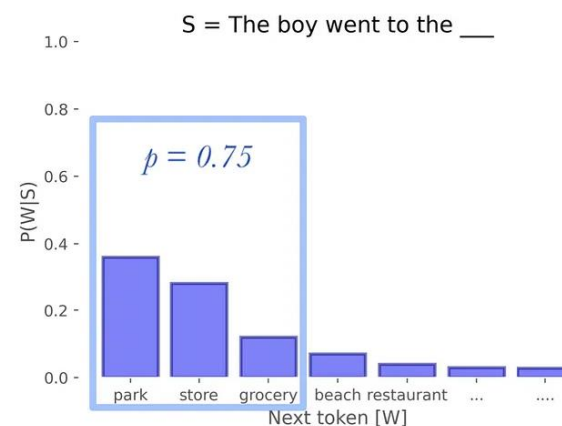
$$w_3 = \arg \max_{w \in V} P(w | w_1, w_2)$$

- Top-k vs top-p sampling:



Top-k sampling

<https://blog.allenai.org/a-gui>

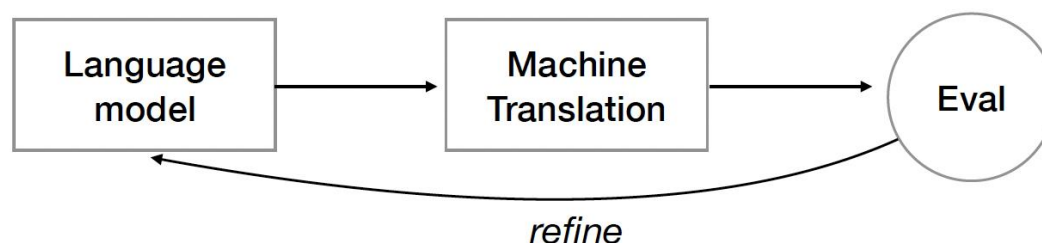


Top-p sampling

[del-sampling-in-allennlp-3b1239274bc3](#)

Evaluating a language model

- Extrinsic evaluation



- Train LM → apply to task → observe accuracy
- Directly optimized for downstream applications
 - higher task accuracy → better model
- Expensive, time consuming
- Hard to optimize downstream objective (indirect feedback)

New Approach to Language Modeling Reduces Speech Recognition Errors by Up to 15%



December 13, 2018

Ankur Gandhe

Alexa

Alexa research

Alexa science

Evaluating a language model

- Intrinsic evaluation

Research process:

- **Train** parameters on a suitable training corpus
 - Assumption: observed sentences $\sim$ good sentences
- **Test** on *different, unseen* corpus
 - If a language model assigns a higher probability to the test set, it is better
- **Evaluation metric** - perplexity!



Perplexity (PPL)

- Measure of how well a LM **predicts** the next word
- For a test corpus with words $w_1, w_2, \dots, w_n$

$$\text{Perplexity} = P(w_1, w_2, \dots, w_n)^{-1/n}$$

$$\text{ppl}(S) = e^x \quad \text{where} \quad x = -\frac{1}{n} \log P(w_1, \dots, w_n) = -\frac{1}{n} \sum_{i=1}^n \log P(w_i | w_1 \dots w_{i-1})$$

Cross-Entropy

• Unigram model: $x = -\frac{1}{n} \sum_{i=1}^n \log P(w_i)$ (since $P(w_j | w_1 \dots w_{j-1}) \approx P(w_j)$)

- Minimizing perplexity $\sim$ maximizing probability of corpus

Intuition on Perplexity (ppl)

If our k-gram model (with vocabulary V) has following probability:

$$P(w \mid w_{i-k}, \dots, w_{i-1}) = \frac{1}{|V|} \quad \forall w \in V$$

what is the perplexity of the test corpus?

A) $e^{|V|}$

B) $|V|$

C) $|V|^2$

D) $e^{-|V|}$

$$\text{ppl}(S) = e^x \quad \text{where} \quad x = -\frac{1}{n} \sum_{i=1}^n \log P(w_i \mid w_1 \dots w_{i-1})$$

Cross-Entropy



Intuition on Perplexity (ppl)

If our k-gram model (with vocabulary V) has following probability:

$$P(w \mid w_{i-k}, \dots, w_{i-1}) = \frac{1}{|V|} \quad \forall w \in V$$

what is the perplexity of the test corpus?

$$\text{ppl}(S) = e^x \quad \text{where} \\ x = -\frac{1}{n} \sum_{i=1}^n \log P(w_i \mid w_1 \dots w_{i-1})$$

A) $e^{|V|}$

B) $|V|$

C) $|V|^2$

D) $e^{-|V|}$

$$\text{ppl} = e^{-\frac{1}{n} n \log(1/|V|)} = |V|$$

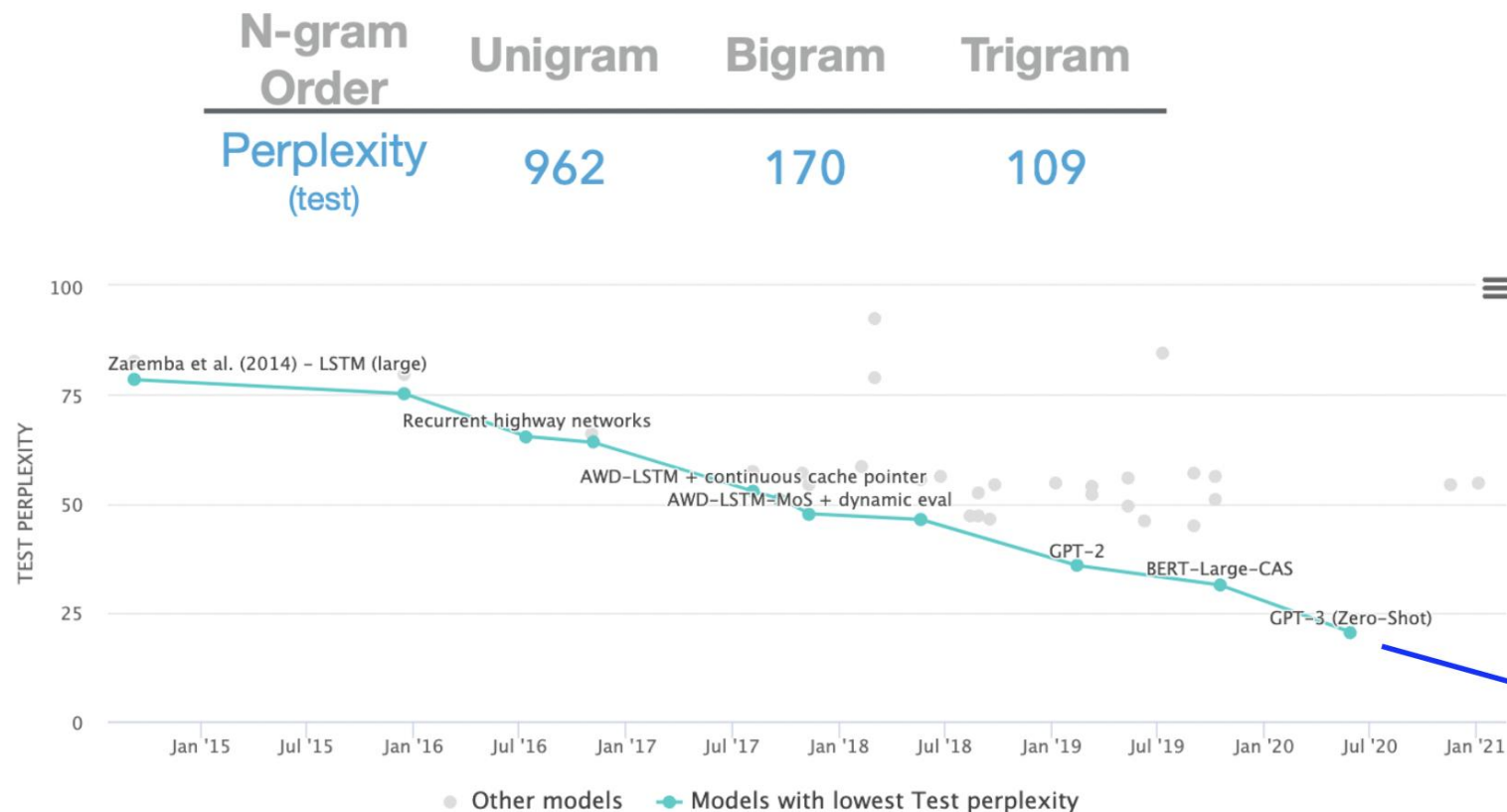
Measure of model's uncertainty about next word (aka 'average branching factor')

branching factor = # of possible words following any word

Perplexity

Training corpus 38 million words, test corpus 1.5 million words, both **WSJ**

ppl càng nh thì oán t tip theo càng tt



GPT-3 175B:
ppl = 20.5

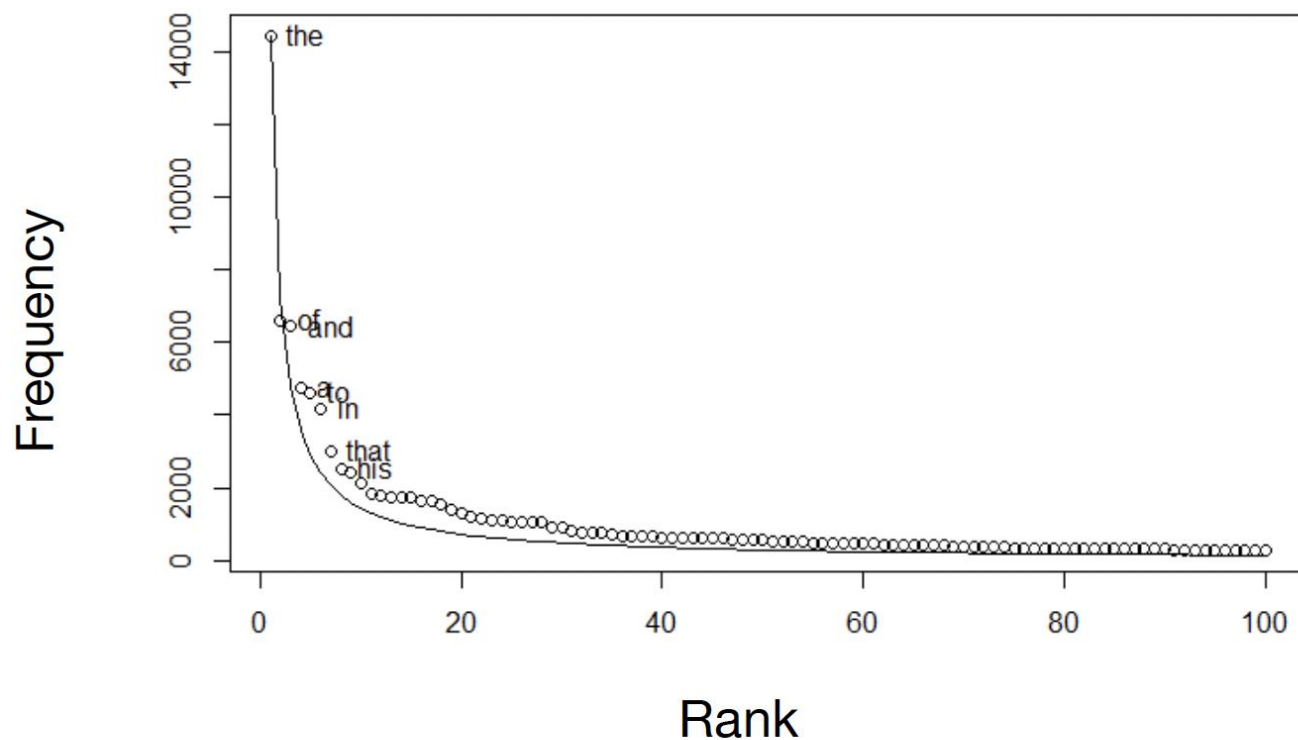
<https://paperswithcode.com/sota/language-modelling-on-penn-treebank-word>

Generalization of n-grams

- Not all n-grams in the test set will be observed in training data
- Test corpus might have some that have zero probability under our model
- **Training set:** *Google news*
- **Test set:** *Shakespeare*
- $P(\text{affray} \mid \text{voice doth us}) = 0 \implies P(\text{test corpus}) = 0$
- Perplexity is not defined.

$$\text{ppl}(S) = e^x \quad \text{where} \\ x = -\frac{1}{n} \sum_{i=1}^n \log P(w_i \mid w_1 \dots w_{i-1})$$

Sparsity in language



$$freq \propto \frac{1}{rank}$$

Zipf's Law

- Long tail of infrequent words
- Most finite-size corpora will have this problem.

- Handle sparsity by making sure all probabilities are non-zero in our model
- **Additive**: Add a small amount to all probabilities
- **Interpolation**: Use a combination of different granularities of n-grams
- **Discounting**: Redistribute probability mass from observed n-grams to unobserved ones

Smoothing intuition

When we have sparse statistics:

$P(w \mid \text{denied the})$

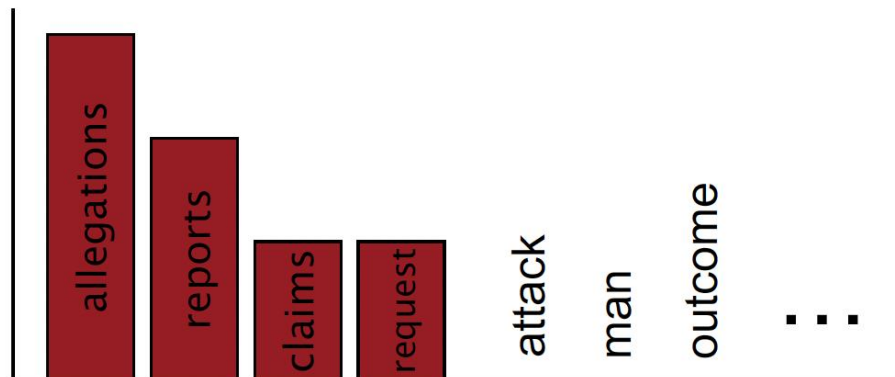
3 allegations

2 reports

1 claims

1 request

7 total



Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

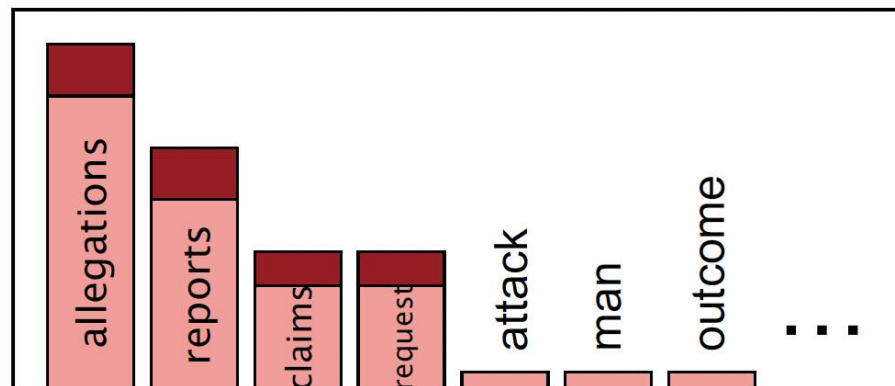
1.5 reports

0.5 claims

0.5 request

2 other

7 total



(Slide credit: Dan Klein)

Laplacian Smoothing

- Also known as add-alpha
- Simplest form of smoothing: Just add α to all counts and renormalize!
- Max likelihood estimate for bigrams:

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

- After smoothing:

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha|V|}$$

Raw bigram counts

- Berkeley restaurant corpus
 - Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

(Slide credit: Dan Jurafsky)

Smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Add 1 to all the entries in the matrix

(Slide credit: Dan Jurafsky)

Smoothed bigram probabilities

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha|V|} \quad \alpha = 1$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

(Credits: Dan Jurafsky)

Linear interpolation

$$\hat{P}(w_i \mid w_{i-2}, w_{i-1}) = \lambda_1 P(w_i \mid w_{i-2}, w_{i-1}) \quad \text{Trigram} \\ + \lambda_2 P(w_i \mid w_{i-1}) \quad \text{Bigram} \\ + \lambda_3 P(w_i) \quad \text{Unigram}$$

$$\sum_i \lambda_i = 1$$

- Use a combination of models to estimate probability
- Strong empirical performance

- Determine some “mass” to remove from probability estimates
- More explicit method for redistributing mass among unseen n-grams
- Just choose an absolute value to discount (usually <1)

Absolute discounting

- Define $\text{Count}^*(x) = \text{Count}(x) - 0.5$

- Missing probability mass:

$$\alpha(w_{i-1}) = 1 - \sum_w \frac{\text{Count}^*(w_{i-1}, w)}{\text{Count}(w_{i-1})}$$

$$\alpha(\text{the}) = 10 \times 0.5/48 = 5/48$$

- Divide this mass between words w for which $\text{Count}(\text{the}, w) > 0$

x	$\text{Count}(x)$	$\text{Count}^*(x)$	$\frac{\text{Count}^*(x)}{\text{Count}(x)}$
the	48		
the, dog	15	14.5	14.5/48
the, woman	11	10.5	10.5/48
the, man	10	9.5	9.5/48
the, park	5	4.5	4.5/48
the, job	2	1.5	1.5/48
the, telescope	1	0.5	0.5/48
the, manual	1	0.5	0.5/48
the, afternoon	1	0.5	0.5/48
the, country	1	0.5	0.5/48
the, street	1	0.5	0.5/48

Absolute discounting

x	$\text{Count}(x)$	$\text{Count}^*(x)$	$\frac{\text{Count}^*(x)}{\text{Count}(x)}$
the	48		
the, dog	15	14.5	14.5/48
the, woman	11	10.5	10.5/48
the, man	10	9.5	9.5/48
the, park	5	4.5	4.5/48
the, job	2	1.5	1.5/48
the, telescope	1	0.5	0.5/48
the, manual	1	0.5	0.5/48
the, afternoon	1	0.5	0.5/48
the, country	1	0.5	0.5/48
the, street	1	0.5	0.5/48

$$\alpha(\text{the}) = 10 \times 0.5/48 = 5/48$$

$$P_{\text{abs_discount}}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) - d}{c(w_{i-1})} \quad \text{if } c(w_{i-1}, w_i) > 0$$

$$\alpha(w_{i-1}) \frac{P(w_i)}{\sum_{w'} P(w')} \quad \text{if } c(w_{i-1}, w_i) = 0$$

Unigram probabilities

When to Use n-gram Models?

- Neural language models achieve better performance, but
- n-gram models are extremely fast to estimate/apply
- n-gram models can be better at modeling low-frequency phenomena
- **Toolkit:** kenlm

<https://github.com/kpu/kenlm>



HUST

Word tokenization

How many words in a sentence?

- "I do uh main- mainly business data processing"
 - Fragments, filled pauses
- "Seuss's **cat** in the hat is different from other **cats**!"
 - **Lemma**: same stem, part of speech, rough word sense
 - **cat** and **cats** = same lemma
 - **Wordform**: the full inflected surface form
 - **cat** and **cats** = different wordforms

How many words in a sentence?

they lay back on the San Francisco grass and looked at the stars and their

- **Type:** an element of the vocabulary.
- **Token:** an instance of that type in running text.
- How many?
 - 15 tokens (or 14)
 - 13 types (or 12) (or 11?)

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, $|V|$ is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^\beta$ where often $.67 < \beta < .75$

i.e., vocabulary size grows with $>$ square root of the number of word tokens

	Tokens = N	Types = $ V $
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13+ million

Corpora vary along dimension like

- **Language:** 7097 languages in the world
- **Variety**, like African American Language varieties.
 - AAE Twitter posts might include forms like "iont" (*I don't*)
- **Code switching**, e.g., Spanish/English, Hindi/English:
 - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:]
 - H/E: dost tha or ra- hega ... dont worry ... but dherya rakhe
["he was and will remain a friend ... don't worry ... but have faith"]
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics:** writer's age, gender, ethnicity, SES

- Every NLP task requires text normalization:
 1. Tokenizing (segmenting) words
 2. Normalizing word formats
 3. Segmenting sentences

1 tách t
2 chuẩn hóa
3 tách câu

Space-based tokenization

- A very simple way to tokenize
 - For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
 - Segment off a token between instances of spaces
- Unix tools for space-based tokenization
 - The "tr" command
 - Inspired by Ken Church's UNIX for Poets
 - Given a text file, output the word tokens and their frequencies



Simple Tokenization in UNIX

- (Inspired by Ken Church's UNIX for Poets.)
- Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | uniq -c
```

Change all non-alpha to newlines

Sort in alphabetical order

Merge and count each type

```
1945 A
 72 AARON
 19 ABBESS
  5 ABBOT
... ..
25 Aaron
 6 Abate
 1 Abates
 5 Abbess
 6 Abbey
 3 Abbot
```

.... ..



The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

```
THE  
SONNETS  
by  
William  
Shakespeare  
From  
fairest  
creatures  
We  
...
```



The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...



More counting

- Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c
```

- Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
8954 d
```

What happened here?

Issues in Tokenization

- Can't just blindly remove punctuation:
 - m.p.h., Ph.D., AT&T, cap'n
 - prices (\$45.55)
 - dates (01/02/06)
 - URLs (<http://www.stanford.edu>)
 - hashtags ([#nlproc](#))
 - email addresses (someone@cs.colorado.edu)
- Clitic: a word that doesn't stand on its own
 - "are" in [we're](#), French "je" in [j'ai](#), "le" in [l'honneur](#)
- When should multiword expressions (MWE) be words?
 - [New York](#), [rock 'n' roll](#)

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     ([A-Z]\.)+          # abbreviations, e.g. U.S.A.
...     | \w+(-\w+)*        # words with optional internal hyphens
...     | \$?\d+(\.\d+)?%?   # currency and percentages, e.g. $12.40, 82%
...     | \.\.\.            # ellipsis
...     | [][.,;"'()?:_-' ] # these are separate tokens; includes ], [
...     , , ,
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```


Tokenization in languages without spaces

Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

How do we decide where the token boundaries should be?



Word tokenization in Chinese

Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")

Each one represents a meaning unit called a morpheme.

Each word has on average 2.4 of them.

But deciding what counts as a word is complex and not agreed upon.



How to do word tokenization in Chinese?

- 姚明进入总决赛 “Yao Ming reaches the finals”
- 3 words?
- 姚明 进入 总决赛
- YaoMing reaches finals
- 5 words?
- 姚 明 进入 总 决赛
- Yao Ming reaches overall finals
- 7 characters? (don't use words at all):
- 姚 明 进 入 总 决 赛
- Yao Ming enter enter overall decision game

Word tokenization / segmentation

So in Chinese it's common to just treat each character (zi) as a token.

- So the **segmentation** step is very simple

In other languages (like Thai, Japanese, Vietnamese), more complex word segmentation is required.

- The standard algorithms are neural sequence models trained by supervised machine learning.



Another option for text tokenization

Instead of

- white-space segmentation
- single-character segmentation

Use the data to tell us how to tokenize.

Subword tokenization (because tokens can be parts of words as well as whole words)



- Three common algorithms:
 - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
 - **Unigram language modeling tokenization** (Kudo, 2018)
 - **WordPiece** (Schuster and Nakajima, 2012)
- All have 2 parts:
 - A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
 - A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,..., a, b, c, d....}

- Repeat:
 - Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
 - Add a new merged symbol 'AB' to the vocabulary
 - Replace every adjacent 'A' 'B' in the corpus with 'AB'.
- Until k merges have been done.

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol ' __ ' before space in training corpus

Next, separate into letters.



BPE token learner

Original (very fascinating) corpus:

low low low low low lowest lowest newer newer newer newer newer newer wider
wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

Corpus

5	l o w _
2	l o w e s t _
6	n e w e r _
3	w i d e r _
2	n e w _

Vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

9 times

Corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

Vocabulary

_, d, e, i, l, n, o, r, s, t, w

Vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

9 times

Corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

Vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

	Corpus	Vocabulary
8 times	5 low__	__, d, e, i, l, n, o, r, s, t, w, er, er__
	2 lowest__	
	6 new er__	Vocabulary
	3 wider__	
	2 new__	__, d, e, i, l, n, o, r, s, t, w, er, er__, ne

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

8 times

Corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

Vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

Vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne, new

Corpus

5 l o w __
2 l o w e s t __
6 n e w e r __
3 w i d e r __
2 n e w __

Final Vocabulary

__, d, e, i, l, n, o, r, s, t, w, er, er__, ne, new, lo, low, newer __, low__

Using BPE for tokenization:

Input: newer__ → **Tokens: newer__**

Input: lower__ → **Tokens: low, er__**

Merge based on the order we learned:

er → er__ → ne → new → newer__

er → er__ → lo → low

Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

A **morpheme** is the smallest meaning-bearing unit of a language

- *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*



Word Normalization

- Putting words/tokens in a standard format
 - U.S.A. or USA
 - uhhuh or uh-huh
 - Fed or fed
 - am, is, be, are

Case folding

- Applications like IR: reduce all letters to lower case
 - Since users tend to use lower case
 - Possible exception: upper case in mid-sentence?
 - e.g., *General Motors*
 - *Fed* vs. *fed*
 - *SAIL* vs. *sail*
- For sentiment analysis, MT, Information extraction
 - Case is helpful (*US* versus *us* is important)



Represent all words as their lemma, their shared root
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')
→ **querer** 'want'
- *He is reading detective stories*
→ *He be read detective story*

- Morphemes:

- The small meaningful units that make up words
- **Stems**: The core meaning-bearing units
- **Affixes**: Parts that adhere to stems, often with grammatical functions

- Morphological Parsers:

- Parse *cats* into two morphemes *cat* and *s*

<https://github.com/sangdv/LLMs>



References

1. Dan Jurafsky and James H. Martin. [Speech and Language Processing](#), 2024.
2. [COS 597G: Understanding Large Language Models, Princeton University](#)
3. [COS 484: Natural Language Processing, Princeton University](#)

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly irregular rings, creating a sense of depth and movement. In the center of this pattern, the word "HUST" is written in a bold, white, sans-serif font.

HUST

Xin trân trọng cảm ơn!